

Linking Code Steps to Analysis: From 60-Second Threshold to Cluster Size Distribution

January 23, 2025

Introduction

This document details how each step in the analysis—from imposing a 60-second threshold to compiling the cluster size distribution—is implemented in your Python code. We correlate each conceptual step to specific lines or functions in your scripts.

1 Step 1: Converting Timestamps and Sorting Pellets

Conceptual Description:

- We take raw timestamps (in hours) and convert them to seconds.
- We sort these timestamps in ascending order to prepare for IPI computation.

Where it happens in your code:

Listing 1: Converting and Sorting Timestamps in `cluster_pellets_by_ipi`

```
def cluster_pellets_by_ipi(pellettimes, ipi_threshold=60):  
    # Convert from hours to seconds and sort  
    pellettimes = np.array(sorted(pellettimes)) * 3600  
  
    # Calculate time differences between consecutive pellets  
    IPIs = np.diff(pellettimes)  
    ...
```

- `pellettimes = np.array(sorted(pellettimes)) * 3600` directly implements:

$$p_i^{(\text{seconds})} = p_i^{(\text{hours})} \times 3600.$$

- Sorting is performed via `sorted(pellettimes)` to ensure correct temporal order.

2 Step 2: Computing the Inter-Pellet Intervals (IPIs)

Conceptual Description:

- We define $\text{IPI}_i = p_{i+1} - p_i$ for consecutive sorted timestamps.

Where it happens in your code:

- In Listing 1, the line:

`IPIs = np.diff(pellettimes)`

executes: $IPI_i = p_{i+1} - p_i$ for each consecutive pair of timestamps in `pellettimes`.

3 Step 3: Applying the 60-Second Threshold to Form Clusters

Conceptual Description:

- If $IPI_i \leq 60$ seconds, pellet $(i + 1)$ belongs to the same feeding cluster as pellet i .
- Otherwise ($IPI_i > 60$), a new cluster begins.

Where it happens in your code:

Listing 2: Threshold-based clustering in `cluster_pellets_by_ipi`

```
def cluster_pellets_by_ipi(pellettimes, ipi_threshold=60):
    pellettimes = np.array(sorted(pellettimes)) * 3600
    IPIs = np.diff(pellettimes)

    clusters = []
    current_cluster = [pellettimes[0]]

    for i, ipi in enumerate(IPIs):
        # If the IPI <= 60s, remain in the same cluster
        if ipi <= ipi_threshold:
            current_cluster.append(pellettimes[i + 1])
        else:
            # Otherwise, start a new cluster
            clusters.append(current_cluster)
            current_cluster = [pellettimes[i + 1]]

    if current_cluster:
        clusters.append(current_cluster)

    return clusters
```

- `ipi_threshold=60` sets the cutoff (60 seconds).
- `if ipi <= ipi_threshold:` implements:

if $IPI_i \leq 60$: same cluster.

4 Step 4: Identifying Cluster Sizes

Conceptual Description:

- Each cluster is a list of pellet timestamps.
- The **size** of a cluster is the number of timestamps in that list.

Where it happens in your code:

Listing 3: Computing cluster sizes in `get_cluster_size_distribution_for_mouse`

```
def get_cluster_size_distribution_for_mouse(pellettimes):
    clusters = cluster_pellets_by_ipi(pellettimes)
    cluster_sizes = [len(cluster) for cluster in clusters]
    return cluster_sizes
```

- The function `cluster_pellets_by_ipi` (Listing 2) returns a list of clusters (each cluster is a list of timestamps).
- `len(cluster)` measures how many pellets (timestamps) are in each cluster, effectively computing the cluster size.

5 Step 5: Accumulating Cluster Sizes Across Mice

Conceptual Description:

- We want to gather cluster-size information for all mice and either sum or average these counts for each cluster size x .

Where it happens in your code:

Listing 4: Accruing cluster size counts across multiple mice

```
def average_cluster_sizes_across_mice(pellettimes_group, max_pellets_per_event=10):
    """
    Calculate the average cluster size distribution across multiple mice.
    """
    total_cluster_counts = np.zeros(max_pellets_per_event)

    # Loop over each mouse
    for pellettimes in pellettimes_group:
        cluster_sizes = get_cluster_size_distribution_for_mouse(pellettimes)
        for size in cluster_sizes:
            if size <= max_pellets_per_event:
                total_cluster_counts[size - 1] += 1
            else:
                total_cluster_counts[-1] += 1

    # Divide by number of mice to get average
    avg_cluster_sizes = total_cluster_counts / len(pellettimes_group)
    return avg_cluster_sizes
```

- `pellettimes_group` is a list of timestamp lists, one per mouse.
- `cluster_sizes = get_cluster_size_distribution_for_mouse(pellettimes)` obtains the cluster sizes for each mouse.
- `total_cluster_counts[x - 1]` accumulates the number of clusters of size x across all mice.

- Finally, `avg_cluster_sizes` divides by the total number of mice, yielding the average frequency for each cluster size—exactly:

$$y(x) = \frac{1}{N} \sum_{i=1}^N n_i(x),$$

where $n_i(x)$ is the count of size- x clusters for mouse i , and N is `len(pellettimes_group)`.

6 Putting it All Together (Conceptual + Code)

1. **Convert and Sort Timestamps:** Implemented at `pellettimes = np.array(sorted(pellettimes)) * 3` in `cluster_pellets_by_ipi` (Listing 1).
2. **Compute IPIs:** `IPIs = np.diff(pellettimes)` in the same function, measuring time gaps between consecutive pellets.
3. **Apply 60s Threshold:** In `cluster_pellets_by_ipi`, if `ipi <= ipi_threshold` decides whether a new cluster should start or continue.
4. **Obtain Cluster Sizes:** `get_cluster_size_distribution_for_mouse` (Listing 3) calls `cluster_pellets_by_ipi` and then uses `len(cluster)` to get each cluster's size.
5. **Aggregate Across Mice:** `average_cluster_sizes_across_mice` (Listing 4) loops over all mice's timestamps, accumulates the counts of cluster sizes, and finally divides by the number of mice for an average distribution.

Conclusion

Each step of your conceptual pipeline—from imposing the 60-second threshold to producing a distribution of cluster sizes—is reflected in specific functions within your Python code. The 60-second rule is enforced in `cluster_pellets_by_ipi`, while the function `get_cluster_size_distribution_for_mouse` translates these clusters into integer sizes. Finally, `average_cluster_sizes_across_mice` aggregates (and averages) those sizes across all selected mice, yielding the *cluster size distribution*.